descope

Applications in Descope
Who calls whom, and why it matters

Inbound (Third-Party and M2M) and Outbound

01 One question sorts every app: who is calling whom?

1

Inbound: Third-Party

Someone else's app calls YOUR API on behalf of one of your users. The user (or their admin) approves exactly what is shared. Think: "Zapier wants to read your HubSpot contacts. Allow?"

2

Inbound: M2M

Someone else's server calls YOUR API with no person in the loop. It proves who it is with a client ID and secret and gets a scoped token. Think: a nightly sync job.

3

Outbound

YOUR app calls someone else's API (Google, Slack, Salesforce) on behalf of your user. Descope connects the account, stores the token, and keeps it fresh.

02 The three apps at a glance

	Inbound: Third-Party	Inbound: M2M	Outbound
Direction	Partner app → your API	Partner server → your API	Your app → partner API
Is a user involved?	Yes, signs in and consents	No	Yes, connects their account
OAuth grant	Authorization code (+ PKCE)	Client credentials	Authorization code, to the provider
Descope acts as	Your authorization server	Your authorization server	Your token vault
What you get	Scoped tokens you issue	Scoped tokens you issue	Provider tokens, auto-refreshed
Everyday example	Zapier connecting to HubSpot	PayPal REST API credentials	Calendly reading Google Calendar

Inbound Apps: Third-Party access



03 Inbound Third-Party: a valet key for your API

You do not hand the valet your house keys, just a key that starts the car. Third-party access works the same way.

What it is

- Your product becomes the place partners "connect to"
- A partner app gets limited, revocable access to a user's data
- No passwords are ever shared with the partner

How it works

- Partner sends the user to your Descope /authorize page
- User signs in and sees a consent screen with the scopes
- Descope issues tokens; the partner calls your API with them

What Descope handles

- OAuth 2.0 / OIDC authorization server endpoints
- Consent screens built in Flows, your branding
- Policy sets the max scopes; users can approve less
- Revoke a partner app at any time

04 You have used this a hundred times

Every "Connect your account" button on a partner site is an inbound third-party app, seen from the side that owns the data.

Zapier → HubSpot

- You build a Zap: new HubSpot contact → Slack message
- Zapier sends you to HubSpot to log in
- HubSpot shows: "Zapier wants to read contacts"
- HubSpot owns the API, so for HubSpot, Zapier is an inbound third-party app

Calendly → Google Calendar

- Calendly needs to see when you are busy
- Google shows a consent screen listing calendar access
- Remove it later in your Google account settings
- For Google, Calendly is an inbound third-party app

Installing an app in Slack

- A workspace admin adds Asana to Slack
- The admin approves the scopes once for the whole workspace
- Every member can use it without re-consenting
- Tenant-level consent, the B2B version of the same idea

05 Inbound Third-Party, step by step

1

Partner asks

The user clicks "Connect" in the partner app. The partner redirects to your Descope /authorize URL with the scopes it wants.

2

User decides

The user signs in with your normal login flow and sees what is being requested. They approve all, some, or none.

3

Descope issues tokens

The partner exchanges a one-time code for an access token and refresh token, limited to the approved scopes.

4

Your API enforces

Your API validates the token and checks scopes on every call. Revoke the app and access stops immediately.

Inbound Apps: Machine-to-Machine (M2M)



06 Inbound M2M: a badge for a server

No browser, no login screen, no person. A backend service proves who it is and gets exactly the access it needs.

What it is

- A trusted server or job calls your API directly
- Used for syncs, batch jobs, integrations, and backend partners
- Access is defined up front, not consented per user

How it works

- Descope gives the client a client ID and client secret
- The server calls /token with grant_type=client_credentials
- It gets a short-lived access token and calls your API

What Descope handles

- Issues and rotates client credentials
- Policies decide which scopes each client gets
- Short-lived tokens, so a leaked token expires fast
- Import existing client credentials when migrating

07 M2M in the real world

If a developer pasted a client ID and secret into a server config, that is machine-to-machine.

PayPal REST API

- Merchants create an app and get a client ID + secret
- Their server trades them for an OAuth access token
- No shopper ever logs in to get that token
- Classic client credentials

Spotify Web API

- An app wants public catalog data: albums, artists, tracks
- It uses the client credentials flow, app identity only
- No user, so no access to anyone's playlists
- Shows how scopes stay narrow without a user

Your B2B customer's backend

- A customer's ERP pushes orders into your API every night
- You give them a Descope M2M client per tenant
- Their job fetches a token, then calls your API
- Rotate or revoke one customer without touching others

08 Third-Party vs M2M: same door, different visitor

Inbound: Third-Party

Acts on behalf of a user

1 user

behind every token

Consent

user or admin approves

- Authorization code flow, PKCE for SPAs and mobile
- Token carries the user's identity
- Use when the partner needs a specific user's data

Inbound: M2M

Acts as itself

0 users

no login, no browser

Policy

scopes set up front

- Client credentials flow
- Token carries the client's identity
- Use for syncs, jobs, and server-to-server integrations

Outbound Apps



09 Outbound: your app reaches out to other services

Inbound is others calling you. Outbound is you calling Google, Slack, Salesforce, or HubSpot for your user.

What it is

- Your user connects their account at another provider
- Your app then calls that provider's API on their behalf
- A token vault, so you never build OAuth plumbing per provider

How it works

- User clicks "Connect Google" in your app (Flow or SDK)
- They consent on Google's own screen
- Descope stores the tokens; your backend fetches them when needed

What Descope handles

- Prebuilt providers: Google, Slack, Salesforce, HubSpot and more
- Secure token storage per user or per tenant
- Automatic refresh before tokens expire
- Progressive scopes: ask for more only when needed

10 Outbound in the real world

Every product with an "Integrations" or "Connected accounts" page is managing outbound tokens.

Calendly

- Connects to your Google or Outlook calendar
- Reads busy times, writes new meetings
- Has to keep the token alive for months
- Without a vault: custom OAuth code per calendar provider

HubSpot

- Connects a rep's Gmail or Outlook inbox
- Logs sent emails to the CRM automatically
- Thousands of reps, one token each
- Refresh failures mean silent data gaps

Canva

- Import images from Google Drive or Dropbox
- Ask for file access only when the user clicks import
- That is progressive scoping in practice
- Start small, request more at the moment of need

11 One handshake, two sides

1

The handshake

A Calendly user clicks "Connect Google Calendar", signs in at Google, and approves calendar access.

2

Google's view: Inbound Third-Party

Google owns the data. Calendly is a third-party app it issued scoped, revocable tokens to.

3

Calendly's view: Outbound

Calendly calls someone else's API. It must store Google's tokens and refresh them. That is an Outbound App.

12 Expose it to your users: drop-in web components

Descope widgets are embeddable web components: add the UI in a few lines instead of building it.

Outbound Applications widget

- A "Connected accounts" page inside your product
- Users connect or disconnect Google, Slack, Salesforce
- Think of Calendly's calendar connections screen
- Tokens land in the Descope vault automatically

Applications Portal widget

- A user-facing list of the apps they can open
- One place for users to launch connected apps
- Embed in your app or host it in the Admin Portal

Access Key widgets

- B2B admins create and rotate their own API keys
- Users can manage personal keys for scripts and jobs
- Self-service M2M setup, no support tickets
- Scoped to the customer's tenant

13 Which app do I need?

If you need to…	Use
Let partners build "Connect to <your product>" integrations	Inbound App, authorization code (third-party)
Let a B2B customer's admin approve an integration for their whole org	Inbound App, tenant-level consent
Let a partner's server or nightly job call your API	Inbound App, client credentials (M2M)
Read a user's Google Calendar, Slack, or CRM data from your app	Outbound App
Do both: partners call you, and you call Google for the same user	Inbound App + Outbound App together

Let others in. Reach out safely.

Descope handles OAuth on both sides, so your team ships integrations, not plumbing.